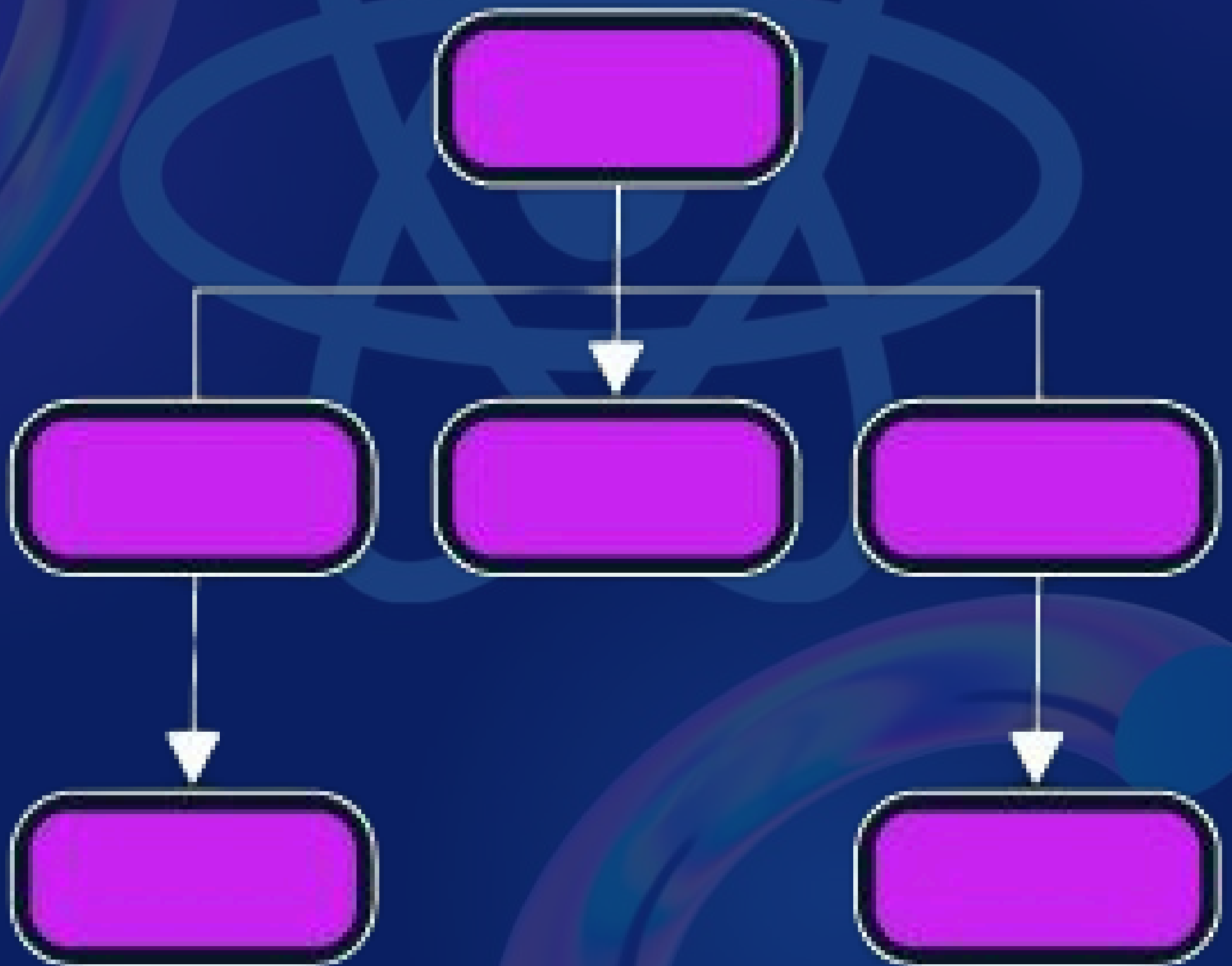


Day 20

Global State Management In React (Part 2)



Combining Context API with Reducer

In our last lesson, we learned about using **Context API** for global state management in React. As much as that is a good approach, it can be further enhanced, especially when working with broader project base, by combining it with **Reducers**.

Combining the **Context API** with the **useReducer** hook in React is a powerful strategy for managing global state in large-scale applications.

It is an alternative to **useState**, useful when the state has multiple sub-values or when the next state depends on the previous one.

This approach allows you to handle complex state logic across multiple components without the need for a third-party library like **Redux**. Although, it is inspired by Redux, but can be used without it

Why Combine Context API and useReducer?

Using the **Context API** alone works well for sharing state across components, but managing complex state transitions can become cumbersome.

The **useReducer** hook simplifies this by encapsulating the logic of state transitions, making your code more predictable and easier to debug.

When you combine them:

- **Context API** provides the mechanism to share state globally.
- **useReducer** manages complex state transitions in a structured way.

This combination allows you to scale state management without adding complexity.

Benefits of This Approach

1. Centralized state management:

By combining **Context API** and **Reducers**, you can manage your application's state from a **single source**. This eliminates the need for "**prop drilling**," where props are passed through many layers of components.

2. Predictable state updates:

Using a **Reducer** ensures that state changes happen in a **predictable** and **controlled** manner. Reducers are **pure functions** that take the **current state** and **an action** to return the **new state**.

This predictable pattern makes it easier to follow and understand how the state is updated, reducing potential errors or unexpected changes.

3. Easier debugging and testing:

With **Reducers**, **state updates** become easier to trace and test since they follow a consistent structure. Each action leads to a well-defined state transition, making it straightforward to debug state changes and test the behavior of your application in isolation.

4. No need for additional libraries:

The combination of **Context API** and **Reducers** provides a built-in, lightweight alternative to Redux. While **Redux** is a robust tool, for smaller applications, this method allows you to manage global state without the need for external libraries, keeping your project simpler and easier to maintain.

Steps to Implement Global State Management with Context API and **useReducer**

- Define the Initial State and Actions
- Create a Reducer Function
- Create a Global Context
- Use the state and dispatch in your components
- Wrap your app with the provider

Step 1. Define the Initial State and Actions

First, you need to define the **initial state** of your global data and the actions that can be performed on it. So, create a state.js file in your “src” folder like this:

```
1 // state.js
2
3 // Define the initial state of your application
4 export const initialState = {
5   user: null,
6   isAuthenticated: false,
7   theme: 'light', // Example of additional state
8 };
9
10 // Define action types
11 export const actionTypes = {
12   SET_USER: 'SET_USER',
13   LOGOUT: 'LOGOUT',
14   TOGGLE_THEME: 'TOGGLE_THEME',
15 };
```

In the above component,

- **Initial State:** Holds the initial values of your state (user, authentication, theme).
- **Action Types:** Constants that define the types of actions that can modify the state. These actions help prevent bugs by avoiding typos in strings.

Step 2. Create a Reducer Function

The reducer is responsible for handling actions and updating the state accordingly.

So, the initial states declared in the state.js file will be updated using the **actionTypes** to get each state.

Now, create another file named reducer.js like this:

```
1 // reducer.js
2
3 import { actionTypes } from './state';
4
5 // Define the reducer function
6 export const reducer = (state, action) => {
7   switch (action.type) {
8     case actionTypes.SET_USER:
9       return {
10         ...state,
11         user: action.payload,
12         isAuthenticated: true,
13       };
14
15     case actionTypes.LOGOUT:
16       return {
17         ...state,
18         user: null,
19         isAuthenticated: false,
20       };
21
22     case actionTypes.TOGGLE_THEME:
23       return {
24         ...state,
25         theme: state.theme === 'light' ? 'dark' : 'light',
26       };
27
28     default:
29       return state;
30   }
31 };
```

takes in the initial state and action for updating the state

initial state accessed

state updated

In the component above:

Reducer Function: Takes the current state and an action, and returns a new state based on the action type.

Switch Statement: Handles different action types. For example, **SET_USER** updates the **user** and **sets the isAuthenticated flag**, **LOGOUT** clears the user, and **TOGGLE_THEME** switches between light and dark modes.

Now, our reducer setup is complete.

Step 3. Create a Global Context

Next, we create a **React context** to **provide** and **consume** the global state.

```
1 // GlobalContext.js
2
3 import React, { createContext, useContext, useReducer } from 'react';
4 import { reducer } from './reducer';
5 import { initialState } from './state';
6
7 // Create a context for global state
8 const GlobalStateContext = createContext();
9
10 // Create a custom provider component
11 export const GlobalStateProvider = ({ children }) => {
12   const [state, dispatch] = useReducer(reducer, initialState);
13
14   return (
15     <GlobalStateContext.Provider value={{ state, dispatch }}>
16       {children}
17     </GlobalStateContext.Provider>
18   );
19 };
20
21 // Custom hook to use the global state context
22 export const useGlobalState = () => useContext(GlobalStateContext);
```

from reducer.js

from state.js

components can now have access to these values

Here,

- **GlobalStateContext:** A context that holds the global state and the dispatch function, which are both made accessible to the component tree.

- **GlobalStateProvider:** A wrapper component that uses `useReducer` to create a state and dispatch function, and passes these to the context provider.
- **useGlobalState:** A custom hook that allows any component in the app to access the global state and dispatch actions.

Step 4. Using Global State in Components

Now, we can use the **global state** and **dispatch** actions in any component.

Let's use a User login and logout as an example. Create a login component (`Login.js/jsx`):

```
1 // Login.js
2
3 import { useState } from 'react';
4 import { useGlobalState } from '../GlobalContext';
5 import { actionTypes } from './state';
6
7 const Login = () => {
8   const { dispatch } = useGlobalState();
9   const [username, setUsername] = useState('');
10
11   const handleLogin = () => {
12     // Dispatch the action to set the user
13     dispatch({
14       type: actionTypes.SET_USER,
15       payload: { name: username },
16     });
17   };
18
19   return (
20     <div>
21       <input
22         type="text"
23         placeholder="Enter username"
24         value={username}
25         onChange={(e) => setUsername(e.target.value)}
26       />
27       <button onClick={handleLogin}>Login</button>
28     </div>
29   );
30 };
31
32 export default Login;
```

accessing the dispatch value using the custom hook from context

USER state is set

In the **Login** component, the **dispatch** function is called with the **SET_USER** action to **update** the **global state** when the **login button** is clicked.

Displaying the User Info: create another component called `UserInfo.js`:

```
1 // UserInfo.js
2
3 import React from 'react';
4 import { useGlobalState } from '../GlobalContext';
5
6 const UserInfo = () => {
7   const { state } = useGlobalState();
8
9   return (
10     <div>
11       {state.isAuthenticated ? (
12         <p>Welcome, {state.user.name}!</p>
13       ) : (
14         <p>Please log in.</p>
15       )}
16     </div>
17   );
18 };
19
20 export default UserInfo;
```

- **state.isAuthenticated**: The component conditionally renders content based on the **isAuthenticated flag** in the global state.

Let's add a global logout feature:

Create a Logout Component:

```
1 // Logout.js
2
3 import React from 'react';
4 import { useGlobalState } from '../GlobalContext';
5 import { actionTypes } from '../state';
6
7 const Logout = () => {
8   const { dispatch } = useGlobalState();
9
10  const handleLogout = () => {
11    // Dispatch the action to log out the user
12    dispatch({ type: actionTypes.LOGOUT });
13  };
14
15  return <button onClick={handleLogout}>Logout</button>;
16 };
17
18 export default Logout;
```

When the **Logout button** is clicked, it dispatches the **LOGOUT** action to reset the user in the global state

Step 5: Wrap your app with the provider

Next is to wrap your app with the **GlobalStateProvider**, you simply need to import the GlobalStateProvider component (which you created earlier) and use it as a wrap around your entire app.

It could be in your **App.js** or **main.jsx** (vite)

```
1 // App.js
2
3 import { GlobalStateProvider } from './GlobalContext'; // Import your GlobalStateProvider
4 import Login from './Login'; // Import the Login component
5 import UserInfo from './UserInfo'; // Import the UserInfo component
6 import Logout from './Logout'; // Import the Logout component
7
8 function App() {
9   return (
10     // Wrap your entire app inside the GlobalStateProvider
11     <GlobalStateProvider>
12       <div className="App">
13         <h1>Global State with Context and useReducer</h1>
14         { /* Components inside this provider will have access to the global state */ }
15         <Login />
16         <UserInfo />
17         <Logout />
18       </div>
19     </GlobalStateProvider>
20   );
21 }
22
23 export default App;
```

- **GlobalStateProvider:** This provider component wraps the entire application, making the global **state** and **dispatch** function available to all nested components (Login, UserInfo, Logout).
- **Children Components:** The **Login**, **UserInfo**, and **Logout** components can now access the global state and dispatch actions because they are nested inside the **GlobalStateProvider**.

In summary, **GlobalStateProvider** encapsulates the whole application, meaning any component within it has access to the global state without needing to manually pass **props** down through multiple levels.

Best Practices for this Combination

1. Separate Concerns:

Keep your context and reducer logic in separate files. This improves maintainability and readability

2. Use Multiple Contexts:

Separate your state into logical domains, and make use of multiple contexts for different parts of your app when necessary. This can improve performance and make your code more modular, also note that too much global state can become difficult to manage.

3. Memoization

Use `React.memo` for components that often receive the same props and for expensive computations. This can help prevent unnecessary re-renders.

Stay tuned for Part 3, where we'll cover the next advanced topic which is the use of Redux state management library to handle complex state logic.



I hope you found this material
useful and helpful.

Remember to:

Like

Save for future reference

&

Share with your network, be
helpful to someone 🙌

Hi There!

Thank you for reading through

Did you enjoy this knowledge?



Follow my LinkedIn page for more work-life balancing and Coding tips.



LinkedIn: Oluwakemi Oluwadahunsi

kodemaven-portfolio.vercel.app